



INSTITUT TECCART 2025

3030 Hochelaga, Montréal, Québec, H1W 1G2

Laboratoire 1

Contexte d'affaire

Eventia Location est une compagnie montréalaise qui loue du matériel pour des événements : systèmes de son, projecteurs, éclairage, tables, chaises et accessoires.

Jusqu'à présent, les employés utilisent des feuilles de calcul et communiquent par courriel. Les erreurs de disponibilité et les doubles réservations deviennent fréquentes. La compagnie demande une application web pour automatiser ces processus d'affaires.

Pour bien comprendre le besoin de cette compagnie, une entrevue a eu lieu entre les analystes fonctionnels de l'équipe de développement et les représentants d'affaires de la compagnie.

Le dialogue suivant résume les points important de ces entrevues:

Analyste : Pouvez-vous décrire votre travail actuel?

Directrice : Lorsqu'un client appelle, nous inscrivons son nom, son courriel et son téléphone dans une feuille de calcul. Nous cherchons ensuite le matériel disponible et nous calculons le prix manuellement.

Analyste : Qu'aimeriez-vous automatiser?

Directrice : Nous voulons enregistrer, consulter, modifier et supprimer les clients. Le courriel d'un client doit être unique.

Responsable de l'inventaire : Je dois pouvoir ajouter du matériel avec un nom, une catégorie, un prix quotidien et une quantité disponible. Je dois aussi le consulter, le modifier et le supprimer.

Analyste : Comment une réservation doit-elle fonctionner?

Préposée : Je sélectionne un client, un matériel, une quantité, une date de début et une date de fin. Le système doit vérifier que le client existe, que le matériel existe et que la quantité demandée est disponible.

Directrice : Une réservation ne doit pas être acceptée lorsque la date de fin précède la date de début ou lorsque la quantité est inférieure à un.

Responsable de l'inventaire : Lorsqu'une réservation est confirmée, la quantité disponible doit diminuer. Lorsqu'elle est annulée, la quantité doit être remise dans l'inventaire.

Comptable : Le prix total correspond au nombre de jours, incluant le premier et le dernier jour, multiplié par la quantité et par le prix quotidien.

Directrice : Après une confirmation ou une annulation, nous voulons conserver une notification. Pour l'instant, elle sera enregistrée dans la base; aucun vrai courriel n'est requis.

Analyste : Qui utilise l'application?

Directrice : Seulement nos employés. L'authentification n'est pas demandée dans cette première version.

Analyste : Que doit afficher l'interface?

Préposée : Quatre sections : clients, matériel, réservations et notifications. Chaque section permet d'afficher la liste des clients, matériaux, réservations et notifications, respectivement, ainsi qu'en ajouter, en modifier ou supprimer.

Travail demandé:

À partir de ce dialogue, l'équipe doit produire les exigences fonctionnelles, et proposer une liste de *user stories* à partir des exigences produits.

Architecture à implémenter

L'application repose sur une architecture services. Chaque service possède sa propre base de données MongoDB et est responsable d'un seul domaine fonctionnel. Les services communiquent exclusivement au moyen d'API REST. Le frontend React agit comme point d'entrée de l'application et interagit directement avec les différents services selon les fonctionnalités demandées par l'utilisateur.

Service Client: ce service est responsable de la gestion des informations relatives aux clients. Il permet de créer, consulter, modifier et supprimer les fiches des clients.

Service matériel: ce service est responsable de la gestion du catalogue des équipements disponibles à la location. Il assure également le suivi de leur disponibilité.

Service réservation: ce service est responsable de la gestion des réservations. Il coordonne les opérations nécessaires à la création et à l'annulation d'une réservation en communiquant avec les autres services lorsque cela est requis.

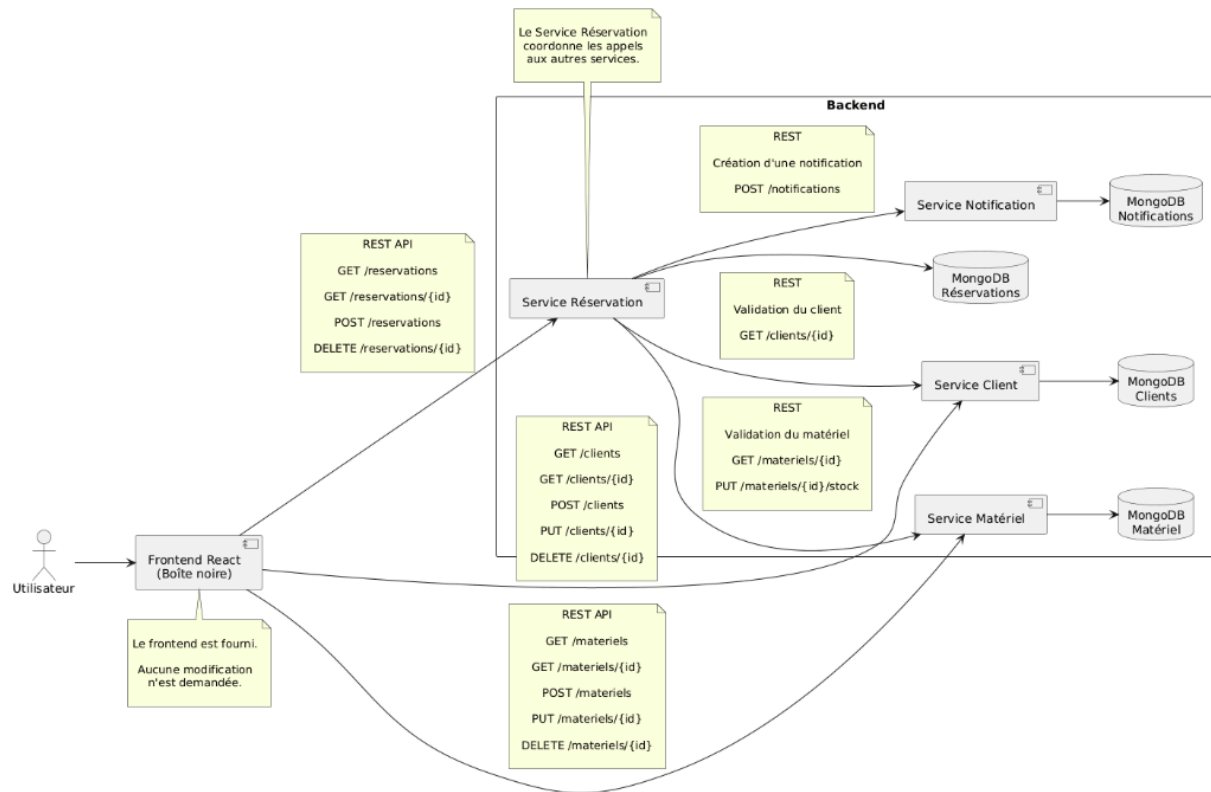
Service réservation: ce service est responsable de la création et de la gestion des notifications générées par l'application lors des différentes opérations métier.

Le *frontend React* est fourni et ne doit pas être modifié. Il communique directement avec les services afin d'exécuter les opérations demandées par l'utilisateur.

Le frontend interagit avec le *service client* pour la gestion des clients, le *service matériel* pour la gestion des équipements, et le *service réservation* pour la gestion des réservations.

Le *service notification* n'est jamais appelé directement par le frontend. Les notifications sont automatiquement déclenchées par les autres services selon les besoins.

La figure suivante résume l'architecture et les composants décrits:



Conception contractuelle

- Les contrats REST à implémenter pour le service des clients et le suivant:
<http://localhost:4001/api/clients>

Méthode	URL	Corps	Réponse attendue
GET	/	Vide	200, tableau des clients
GET	/:id	Vide	200 + client/404 si le client est introuvable
POST	/	{name, email, phone}	201 + le client crée
PUT	/:id	{name, email, phone}	200 + client modifie
DELETE	/:id	Vide	204

- Les contrats REST à implémenter pour le service des équipements sont les suivants:
<http://localhost:4002/api/equipements>

Méthode	URL	Corps	Réponse attendue
GET	/	Vide	200, tableau des équipements
GET	/:id	Vide	200 + client/404 si l'équipement est introuvable
POST	/	{name, category, dailyPrice, availableQuantity}	201 + l'équipement crée
PUT	/:id	{name, category, dailyPrice, availableQuantity}	200 + client modifie
PUT	/:id/reserve	{quantity}	200 + soustraction de la quantité, erreur si insuffisante
PUT	/:id/release	{quantity}	200, remet la quantité
DELETE	/:id	Vide	204

- Les contrats REST à implémenter pour le service des réservations sont les suivants:
<http://localhost:4003/api/reservations>

Méthode	URL	Corps	Réponse attendue
GET	/	Vide	200, tableau des réservations
POST	/	{clientId,equipmentId, quantity,startDate,end Date}	201, réservation enrichie avec noms, prix total et statut
PUT	/:id/cancel	Vide	200, réservation avec statut CANCELLED
PUT	/:id	{name, category, dailyPrice, availableQuantity}	200 + client modifie
PUT	/:id/reserve	{quantity}	200 + soustraction de

			la quantité, erreur si insuffisante
PUT	/:id/release	{quantity}	200, remet la quantité
DELETE	/:id	Vide	204

Lors du **POST**, ce service appelle **GET** /api/clients/:id, **GET** /api/equipments/:id, **PUT** /api/equipments/:id/reserve et **POST** /api/notifications.

Lors de l'annulation, il appelle **PUT** /api/equipments/:id/release et **POST** /api/notifications

- Les contrats REST à implémenter pour le service des notifications sont les suivants:
http://localhost:4004/api/notifications

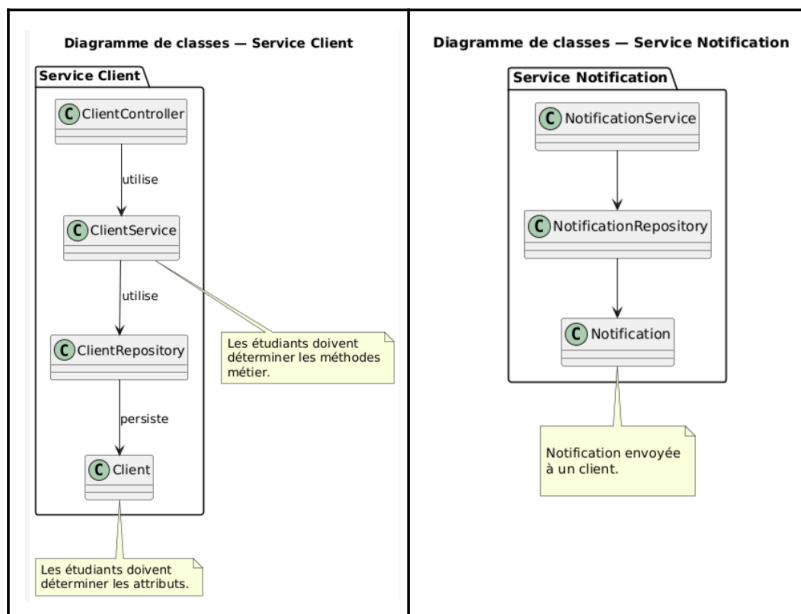
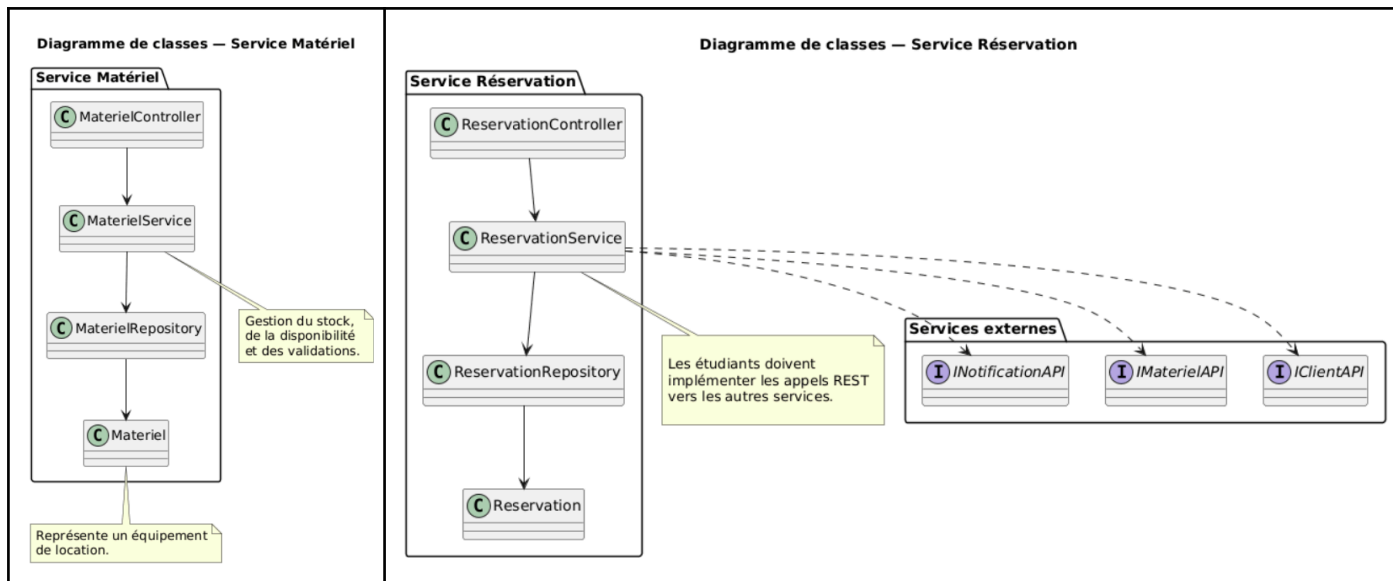
Méthode	URL	Corps	Réponse attendue
GET	/	Vide	200, notifications de la plus récente à la plus ancienne
POST	/	{recipient, message, type}	201, notification créée

Travail demandé:

Il est demandé d'implémenter les APIs REST pour les services tel qu'il est décrit dans chacun des tableaux.

Conception détaillée

Dans cette partie du travail, un squelette de classe décrivant la structure de classe de chaque service est fourni, comme montré dans la figure suivante:



Travail demandé

Il est demandé par la suite d'identifier les méthodes et les attributs de chacune des classes, les documenter dans un diagramme de classe en utilisant UML, et les implémenter selon la description fournie dans le code source du projet.

Stratégies de branches Github

Voici les différentes branches impliquées dans le projet:

- main : version stable et démontrable (branche protégée).
- develop : branche d'intégration de l'équipe.
- feature/client-service : travail du responsable du service client.
- feature/equipment-service: travail du responsable du matériel.
- feature/reservation-service: travail du responsable des réservations.

- feature/notification-service: travail du responsable des notifications.
- docs/analyse-conception : Liste des exigences, des user stories, et les diagrammes raffinés.

Aucun push direct vers main par les développeurs, chaque fonctionnalité passe d'abord par une pull request vers la branche develop, puis au moins un autre membre révise la PR, l'auteur corrige les commentaires. L'intégration finale se fait par PR de develop vers main.

Travail demandé:

Un membre de l'équipe se charge de créer un dépôt public sur la plateforme github, en créer les branches décrites précédemment, puis en inviter des collaborateurs de l'équipe.

Livrables attendus:

1. La liste des exigences fonctionnelles.
2. La liste des user stories.
3. Des diagrammes UML documentant les classes de chacun des services avec leurs attributs et méthodes.
4. Lien github montrant les commits de chaque membre du groupe, et les différents pull requests. La version initiale du code source sera fournie avec l'énoncé, et il est demandé de compléter cette version pour la rendre fonctionnelle.
5. À la fin du projet, chaque équipe devra répondre à un questionnaire portant sur les choix de conception, l'architecture, les contrats REST, l'implémentation et les interactions entre les services. Ce questionnaire servira à évaluer la compréhension de la solution développée, et chaque équipe doit répondre de façon cohérente avec ses livrables.

Grille d'évaluation

Critère	Bien	Acceptable	Insuffisant
CRITÈRE A: Élicitation des exigences fonctionnelles et compréhension des besoins (10%)	Les besoins du client sont très bien compris. Les exigences fonctionnelles couvrent l'ensemble des fonctionnalités demandées. Les ambiguïtés sont correctement levées et la description reflète fidèlement les attentes du client.	Les besoins principaux sont compris, mais certaines exigences sont incomplètes ou imprécises.	Les besoins sont mal interprétés. Plusieurs exigences importantes sont absentes ou erronées.

CRITÈRE B: Spécification des user stories (10%)	Les user stories sont cohérentes et suffisamment détaillées pour guider la conception.	Les User Stories sont généralement correctes mais certains critères d'acceptation sont incomplets ou peu précis. Quelques incohérences subsistent.	Les User Stories sont incomplètes ou mal formulées. Les critères d'acceptation sont absents ou ne permettent pas de vérifier les fonctionnalités.
CRITÈRE C: Implémentation de la conception haut niveau (contrats REST) (20%)	Les responsabilités des services sont bien implémentées dans le code. Les contrats REST sont complets, cohérents et permettent une communication efficace entre les composants.	L'architecture est fonctionnelle mais certaines responsabilités sont mal implémentées ou certains contrats REST sont incomplets ou imprécis.	L'architecture est incohérente ou ne respecte pas la spécification de l'énoncé. Les contrats REST sont absents, incomplets ou incompatibles entre eux.
CRITÈRE D: Implémentation de la conception détaillée (Diagrammes UML) (20%)	Les diagrammes de classes UML représentent fidèlement la solution. Les classes, attributs, méthodes, relations et multiplicités sont implémentées de façon cohérente avec les exigences fonctionnelles.	Les diagrammes UML, ainsi que l'implémentation sont globalement corrects mais présentent quelques omissions ou erreurs mineures dans les attributs, méthodes ou relations.	Les diagrammes UML sont incomplets, incohérents ou ne permettent pas de guider correctement l'implémentation. Incohérence entre les diagrammes UML fournis et l'implémentation.
CRITÈRE E: Intégration des différents services (5%)	Les services communiquent correctement entre eux et le frontend fonctionne entièrement sans modification.	L'intégration est partiellement fonctionnelle. Certaines interactions entre les services présentent des erreurs mineures.	Les services ne s'intègrent pas correctement ou le frontend ne peut pas fonctionner avec les services développés.

CRITÈRE F: Utilisation de Git et GitHub (20%)	La stratégie de branches est respectée. Les commits sont fréquents et significatifs. Les Pull Requests et les fusions démontrent une collaboration efficace entre les membres de l'équipe.	Les principales fonctionnalités de Git sont utilisées mais la stratégie de collaboration est appliquée de manière incomplète.	La collaboration Git est insuffisante. Les branches, les commits ou les Pull Requests sont absents ou mal utilisés.
CRITÈRE G: Reponses au questionnaire (15%)	Les réponses sont précises et reflètent le travail fait	Les réponses sont mal formulées mais en cohérence avec le travail fait	Incohérence entre les réponses fournies et les livrables soumis.